

Trabalho Prático — API REST com Spring Boot

Tema: Sistema de Gerenciamento de Biblioteca (Autor e Livro)

1. Objetivo

Este trabalho tem o objetivo de fixar os conceitos de Programação Orientada a Objetos aplicados ao desenvolvimento de APIs REST com Spring Boot, seguindo a mesma arquitetura em camadas trabalhada em sala de aula (Entity, Repository, Service e Resource).

2. Domínio do problema

O sistema deverá gerenciar uma biblioteca simples, composta pelas entidades Autor e Livro, relacionadas da seguinte forma:

- Um Autor pode ter vários Livros (1:N)
- Um Livro pertence a um único Autor (N:1)

Entidade Autor

Atributo	Tipo	Observação
id	Long	gerado automaticamente
nome	String	obrigatório
nacionalidade	String	obrigatório
dataNascimento	LocalDate	obrigatório
livros	List<Livro>	mapeado pelo relacionamento (mappedBy)

Entidade Livro

Atributo	Tipo	Observação
id	Long	gerado automaticamente
titulo	String	obrigatório
isbn	String	obrigatório e único
anoPublicacao	Integer	não pode ser maior que o ano atual
numeroPaginas	Integer	deve ser maior que zero
autor	Autor	relacionamento @ManyToOne

3. Estrutura de pacotes

Seguir exatamente a mesma organização usada em sala:

```

br.edu.suainstituicao.biblioteca
├── entities
│   ├── Autor.java
│   └── Livro.java
├── repositories
│   ├── AutorRepository.java
│   └── LivroRepository.java
├── services
│   ├── AutorService.java
│   ├── LivroService.java
│   └── exceptions
│       └── ResourceNotFoundException.java
└── resources
    ├── AutorResource.java
    └── LivroResource.java

```

4. Camadas a implementar

Entities: classes anotadas com `@Entity`, `@Id`, `@GeneratedValue`, e o relacionamento mapeado com `@OneToMany`/`@ManyToOne`.

Repositories: interfaces que estendem `JpaRepository<Entidade, Long>`. Deve existir pelo menos um método de consulta personalizado, por exemplo:

```
List<Livro> findByAutorId(Long autorId);
```

Services: contêm as regras de negócio (validações, lançamento de exceções) e fazem a ponte entre `Resource` e `Repository`.

Resources: controllers REST responsáveis por receber as requisições HTTP e devolver as entidades, assim como no exemplo de sala.

5. Endpoints obrigatórios

AutorResource

Método	Endpoint	Descrição
GET	/autores	Lista todos os autores
GET	/autores/{id}	Busca um autor pelo id
POST	/autores	Cadastra um novo autor
PUT	/autores/{id}	Atualiza um autor existente
DELETE	/autores/{id}	Remove um autor

LivroResource

Método	Endpoint	Descrição
GET	/livros	Lista todos os livros
GET	/livros/{id}	Busca um livro pelo id
POST	/livros	Cadastra um novo livro

PUT	/livros/{id}	Atualiza um livro existente
DELETE	/livros/{id}	Remove um livro
GET	/livros/autor/{autorId}	Lista os livros de um autor específico (endpoint personalizado)

6. Requisitos técnicos

- Java + Spring Boot (mesma versão usada em sala).
- Banco de dados PostgreSQL.
- Build com Maven.
- Tratamento de exceções com retorno de códigos HTTP corretos (200, 201, 204, 404, 400 conforme o caso).
- Uso correto dos verbos HTTP e dos status codes (ex.: POST retorna 201 Created, DELETE retorna 204 No Content).

7. Entrega

- Repositório Git (GitHub/GitLab) com o código-fonte completo.

8. Evidências de teste

- Prints do Postman/Insomnia ou arquivo de coleção exportado, demonstrando o funcionamento de todos os endpoints obrigatórios.

9. Critérios de avaliação

Critério	Peso
Modelagem correta das entidades e do relacionamento	25%
Implementação completa das camadas (entity, repository, service, resource)	30%
Funcionamento correto de todos os endpoints obrigatórios	25%
Tratamento de exceções e códigos HTTP adequados	15%
Evidências de teste	5%